Spam Detection in SMS Messages With NLP

Course : DLBAIPNLP01 - Project NLP
Task 2 : Text Classification for Spam Detection
Nathan Mersha Degineh
102303079
Submitted Date : 19/10/2026
Repository : https://github.com/nathan-mersha/Project_nlp

Table of Contents

Table of Contents **2**
1 Introduction **3**
1.1 Background and Problem 3
1.2 Objective 3
1.3 Tools Used 4
1.4 Structure of this report 4
2 Background **5**
2.1 Text Processing 5
2.2 Text Encoding with TF-IDF 5
2.3 Multinomial Naive Bayes 6
2.4 Evaluation Metrics 6
3 Project Planning, Resources & Data **7**
3.1 Phases 7
3.2 Resources and Tools 8
3.3 Data Collection 8
3.4 Risk Analysis 9
4 Implementation **10**
4.1 Structure 10
4.2 Pre-Processing 10
4.3 Data Split and Encoding 11
4.4 Training with Growing Amounts of Data 11
4.5 New Message Classification 11
5 Results **12**
5.1 Different training set results 12
5.2 Confusion Matrix for the final model 13
5.3 Analyzing the result 13
5.4 Requirement and Implementations Checklist 14
6 Final thoughts and Conclusion **15**
6.1 Reflection on the approach I took 15
6.2 Limitations 15
6.3 Improvements 15
6.4 Conclusion 16
Reference **17**

1 Introduction

1.1 Background and Problem

Spam messages are unwanted messages that are sent in mass to a lot of people at the same time.Most of these spam messages advertise products and are harmless. However some of them are used for theft and fraud. For example some spam messages are designed to steal personal data or money in some way, these are usually called phishing attacks. Spam messages are found frequently in emails as compared to SMS because emails are free to send, SMS spam messages are very intrusive because they arrive directly on the users phone and they usually get more attention when compared with Email spams and users tend to trust them more than emails (Almeida et al., 2011).

To check every message manually by hand is impossible. So we need an automatic system that does this for us. This system will read the message and decide if it's spam or a normal message. In this report, I refer to the normal messages as legitimate or ham. In the field of NLP this is one of the most common tasks : a piece of text is assigned to one of a fixed set of categories, in this case spam or ham (Jurafsky & Martin, 2013).

1.2 Objective

The objective of this project is to build a very simple NLP system that classifies these incoming messages as spam or legitimate (ham) . Based on the requirement in the task, this system has to do the following.

1. Use a dataset of messages that are labeled spam or legitimate (ham)
2. Clean the text by removing stop words and by lemmatizing the words and encode the texts as numbers for a machine learning process.
3. Train a supervised machine learning model on the labeled data
4. Evaluate the created model on a different separate test data
5. Classify new messages as spam or ham using a terminal interface.
6. The system is also evaluated first with a small amount of training data and then with more progressively (with 3 steps) this will show how the results change when more training data is available

1.3 Tools Used

This project is similar in terms of pipeline with most ML processes, the steps implemented in this pipeline are :

- Data collection
- Preprocessing
- Encoding
- Training
- Use/Evaluation

The SMS spam collection from the UCI Machine Learning Repository is used in this project for its dataset. Then the texts are cleaned with the NLTK library and encoded with TF-IDF. In this project I used the Multinomial Naive Bayes classifier as the model because of its speed and simplicity and its a widely known model for spam filtering (Sahami et al., 1998). The code is written in Python and uses SciKit learn and the evaluation metrics (Pedregosa et al., 2011). The code can be found in this public gitHub repo https://github.com/nathan-mersha/Project_nlp

1.4 Structure of this report

I have structured this report in six chapters. The first chapter is an introductory chapter explaining the objective of the projects, background and problems and the tool used. The next chapter (chapter two) will explain the theory of the methods that I used for this project. Chapter three explains the plan and the data used for this project.Chapter four explains how I implemented the system. Chapter five shows the result and analysis. And finally in chapter six I try to reflect on the project over all, what the implementations limitations and improvements could be.

2 Background

Text classification means that a program assigns a label to a text. In supervised learning this process is learned by the program from an example, in which the correct label is known, and then based on this training it predicts the label of a new unseen text (Jurafsky & Martin, 2013). Spam classification has a binary output, a text is either spam or legitimate.

2.1 Text Processing

Text preprocessing makes the text cleaner and uniform (Bird et al., 2009). These texts processing steps include :

- **Lowercasing** : “HI” and “hi” becomes the same word
- **Tokenization** : The texts are split into single words or symbols called tokens
- **Removing Stop Words** : Removing frequently appearing words like, “the”, “and” “is” because their meaning is usually little
- **Stemming and Lemmatization** : In stemming words are converted to their base form. “Winning” becomes “win” and in lemmatization the program uses a dictionary and returns real word changes and not just cut off endings so words like **“messages”** become **"message"** for example. In this project I have used lemmatization and not stemming because it returns real words.

2.2 Text Encoding with TF-IDF

Because a model needs numbers and not words TF-IDF turns a message into a vector with one value per word of the vocabulary (Manning et al., 2008). A word gets high value if it shows up often in the message but rarely in other messages (the cleaning up of the texts in the first step helps us remove stop words in these calculations so they won't be considered). Very common spam words like, “prize” or “claim” get a high value (weight), but a word that appears everywhere gets a low value (weight). Here is what the basic formula looks like, where N is the number of messages and df(t) is the number of messages that contain the word t. SciKit learn uses a slightly modified version, where the idf part is ln((1 + N) / (1 + df(t))) + 1, so that a word that appears in every message does not get a weight of zero (Pedregosa et al., 2011)

$$tf\text{-}idf(t, d) = tf(t, d) \times log(N / df(t))$$

2.3 Multinomial Naive Bayes

Naive Bayes is a classifier that works using simple probabilities. It is based on Bayes theorem. When we get a new message this model checks how likely a message is a spam or a ham and picks the one with the higher probability. For the Naive Bayes to do this it checks two things from the training data. The first is, how common spam and ham messages are (called the prior probability) and how often each word shows up in a spam and in ham messages. It is called “naive” because it evaluates / treats each word as if the other words have nothing to do with the other texts. Of course real world language does not work like this, but this method gives good results on text evaluation (Manning et al, 2008). Naive Bayes is really fast, works really well with little data set (in my example for example the data set is less than 6k messages) and in most cases it is used as a starting point for spam filtering (Metsis et al. 2006). There are more advanced models like SVM (Support Vector Machines) or deep learning that are more accurate than Naive Bayes, but they need more computing power, are complex and harder to explain after the fact why a certain message is classified as a spam or ham (Elakkiya et al. 2021; Huan et al. 2022)

2.4 Evaluation Metrics

In this project spam will be treated as a positive class. Based on this every prediction will be one of the four groups :

1. True Positive : Spam that was found
2. False Positive : A normal message wrongly marked as a spam
3. True Negative : A normal message correctly identified as a normal message
4. False Negative : A spam message that was missed

Using these four classifications and their values we can now calculate the following metrics :

- Accuracy = (TP + TN) / All Messages : How many of all predictions were correct
- Precision = TP / (TP + FP) : From all messages marked as spam how many of them really are?
- Recall = TP/ (TP+FN) : From all real spam messages, how many of them did the classifier really find?
- F1 Score = 2 x Precision x Recall / (Precision + Recall) : A single number that combines both precision and recall.

If we use accuracy alone, we will not get a full picture. For example if 90 % of the messages are legitimate. A model that blindly says legitimate will have a score of 90 %. That is why we use the other metrics, to accurately and precisely evaluate the model.

3 Project Planning, Resources & Data

3.1 Phases

The work of this project was split into six phases (Table 1). I planned, developed, tested and created this report according to the assignment instruction for this course. Here are the project phases.

Phase	**Main Tasks**	**Results**
Planning	Picked task 2 from the list of projects of the course task, selected dataset and tools.	Project plan completed
Data collection	Downloaded the SMS spam collection set. Setup my repo and initialized the python virtual env	Raw data and project init completed.
Preprocessing	Write the clean text function	Texts successfully cleaned up.
Training	Split the data, Encoded text with TF-IDF, train Naive Bayes on 10, 50 and 100% of the training data, incrementally	Saved the model. Got metrics.csv and metrics.png
Use	Write a predict command and test it with new text message and hooked it up with the CLI	Any user can use this on the CLI directly to predict messages.
Documentation	Created readme and this report	Github repo and Report pdf completed.

Table 1 : Project Phases

3.2 Resources and Tools

All tools I used in this project are free and open source (Table 2). The software can run on any computer and setting up takes less than a minute. No special hardware is needed to run this, for example a GPU. The exact library versions are saved in the requirements.txt so that anyone who replicates it will get the exact result every time.

Tool	**Version**	**Purpose**
Python	3.13	Programming language of choice, used due to extensive support for data processing and ML related tasks.
Pandas	3.0	To load the data
NLTK	3.10	Tokenization, Stop word lists, Lemmatization
SCIKIT Learn	1.9	TF-IDF, Naive Bayes
MatPlotLib	3.11	To draw the graph of the results
JobLib	1.6	To save and load the training model during new evaluations
Git and Github		To store the code
VS Code		Code editor of choice.

Table 2 : Tools and Libraries

3.3 Data Collection

This project uses the SMS Spam Collection (Almeida & Hidalgo, 2011) from the UCI Machine Learning repo. The data contains 5,574 English SMS messages collected from several public sources. Each message is labeled as ham or spam (Almeida et al., 2011) and saved in its own line. The task suggests the UCI Spambase data set as an example. But I did not use that because the spambase does not have the original texts of the emails (Hopkins et al. 1999) and because the task required removing stop words and lemmatizing the word (over all cleaning of the text) I needed a dataset that has the raw text, that is why I chose the SMS Spam Collection instead.

Class	Messages count	Percentage
Legitimate (Ham)	4,825	86.6
Spam	747	13.4
Total	5,572	100

Table 3 : SMS Spam Collection Data Sets

3.4 Risk Analysis

Before starting the implementation there are some risks identified and how to handle them, these are listed below :

Risk	Effects	Counter Measures
Imbalanced classes	The model prefers the ham data and sometimes misses the spam, so the accuracy values as mentioned previously look better than the model really is	Report precision, recall and F1 score and not just the accuracy
Data Leakage	Information from the test set has been used in the training, so the results look better than they really are.	TF-IDF vectorizer is fitted on the training data only
Results may not be replicable	Different results on every run or on another computer	Fixed random seed of 42, pinned all used libraries to specific versions
Missing NLTK resources	The script fails on a new computer	Script downloads the needed NLTK data automatically

Table 4 : Risks and Counter measures

4 Implementation

4.1 Structure

The entire code is a single python script named : spam_detector.py. It has two main commands : train and predict. The data is stored in the directory data/raw. The trained model and vectorizer is saved in models directory. The result of the report is stored in the reports directory.

4.2 Pre-Processing

The function “cleanup_txt” cleans each message. It changes all the letters in lower case, splits the messages into a single word (tokens) with NLTK word tokenizer. It also removes all common English stop words (like : the, a, and, …) from the NLTK list. It then converts all remaining words into its base form with WordNet lemmatizer (like running will be run) and then puts all the words back together into one string. Examples of this process can be seen in the table below.

Original Message	**After Cleanup**
Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question …	free entry 2 wkly comp win fa cup final tkts 21st may 2005. Text fa 87121 receive entry question …
Go until jurong point, crazy…Available only in bugis n great world la e buffet…	go jurong point, crazy … available bugis n great world la e buffet…

Table 5 : Example of preprocessing

As illustrated in the results here all stop words like : “in” , “a” and “to” are removed. All the words are converted to lowercase. The punctuations still stay in the text but the TF-IDF tokenizer ignores them. The stop word list and lemmatizer are created just once when the script starts. In my first implementation however, I was creating them for every message every time which made my script slower (this has been fixed).

4.3 Data Split and Encoding

The labels of spam and ham are numbers, spam is 1 and ham is 0. The data is then split into 80% for training and 20% for testing. The split between training and testing is fair, both the training and testing data sets have approximately equal share of spams and hams. A fixed state of 42 makes sure the split is the same every time the training is run on both my device and on someone else's computer. And since the model never sees the test messages during training, the test results show how well the model generalizes, which means how well the model works on new messages it has never seen before.

4.4 Training with Growing Amounts of Data

To see how the system performs on different quantities of data sets, the Naive Bayes is trained three times, on 10, 50 and 100 percent of the training set. All the three models are tested on the same test set which means now we can clearly compare their results, which in turn means that we can see how different percentages affect the overall quality of each model. The comparison results are saved in the reports/metrics.csv file and the visual comparison is saved at reports/metrics.png.

4.5 New Message Classification

I have created a predict new messages command, this loads the already saved model and vectorizer, cleans the new message with the cleaner function and then predicts whether it's spam or legitimate.

New message	**Output**
WINNER! You have won a free 1000 euro prize. Call 123456789 now to claim.	Spam
Congratulations! You've been selected for a free cruise. Reply YES to claim!	Spam
Are we meeting for lunch?	Legitimate
Pick up the diaper on your way	Legitimate

Table 6 : New message classification results

5 Results

5.1 Different training set results

In the table and figure below, it can be seen how the three models performed differently on the test sets :

Training %	Messages	Spams	Accuracy	Precision	Recall	F1
10	446	65	0.879	1.000	0.094	0.172
50	2228	293	0.949	1.000	0.617	0.763
100	4457	598	0.964	1.000	0.732	0.845

Table 7 : Results

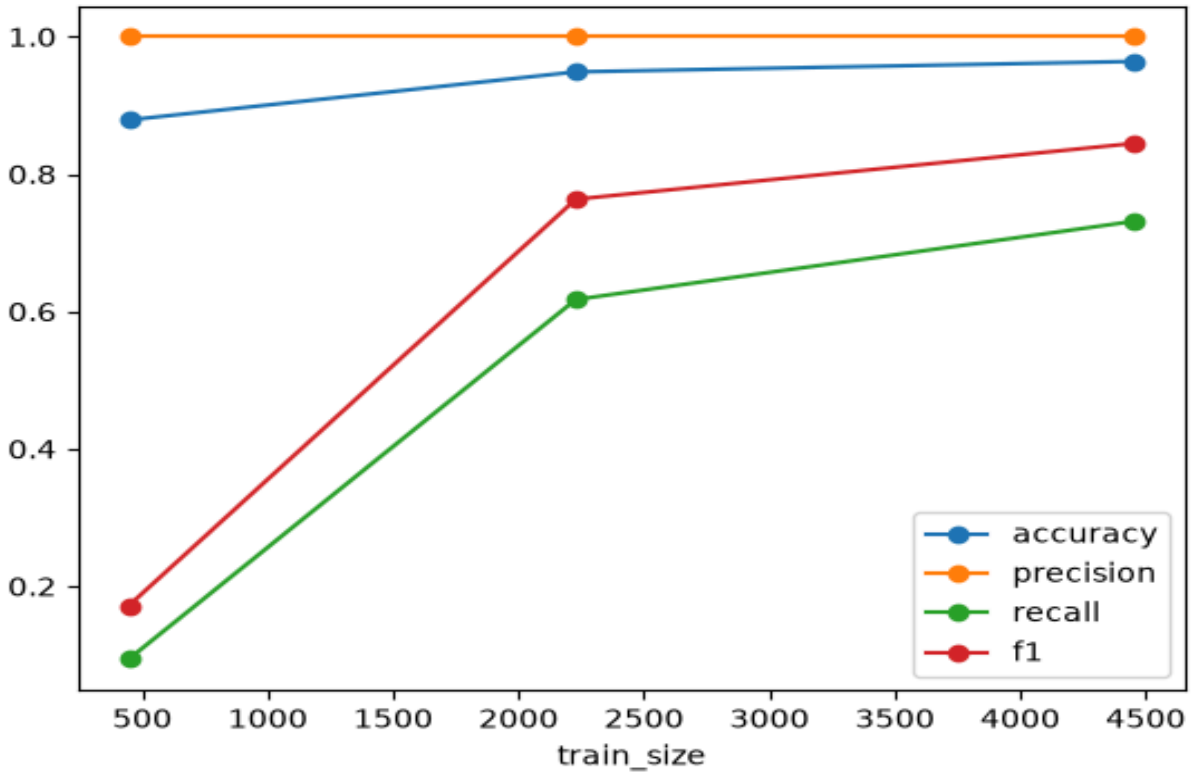


Figure 1 : Accuracy, Precision, Recall and F1 Score by different training sizes

5.2 Confusion Matrix for the final model

The table below shows the confusion matrix of the model that was trained on the 100 percent of the training set, which is 4457 messages.

	Predicted Legitimate	**Predicted Spam**	**Total**
Actual Legitimate	966 (True Negative)	0 (False Positive)	966
Actual Spam	40 (False Negative)	109 (True Positive)	149
Total	1006	109	1,115

Table 8 : Confusion Matrix for the final model

From 1,115 text messages, 1,075 of them were classified correctly (96.4 %). All 40 mistakes are the same. They are spam messages that the model thought was legitimate. Not a single legitimate message was classified as spam.

5.3 Analyzing the result

The model is very careful because it marks a message as spam only when it's very sure. So actual legitimate messages are not blocked, however there are some spam messages that are marked as legitimate. From what is seen there are some reasons for this. The first is the data is not equal for both spam and legitimate. Almost 87 percent of the training messages are legitimate. Another reason that is contributing to this slip up is since there are very few spam messages the model learns what words are in spam messages only from these few examples. But as we have seen from training on different data set this improved the more data we give it (the recall value)

Even though it's not ideal, this behaviour is actually preferable for the user. This means important personal messages never end up in a spam folder but once in a while the user might get a spam message in their main inbox. And according to the different datasets provided we can clearly see that the more data we give it the better the model becomes which means more data would actually improve it even further.

5.4 Requirement and Implementations Checklist

In the table below I try to demonstrate what has been achieved in this project according to the task.

Requirement	**Implementations**	**Status**
Labeled dataset of messages	SMS labeled messages collected, around 5,572 messages (original messages were 5,574 but few lines with quotation marks merged with the next line)	Done
Stop word removal, lemmatization and encoding	NLTK stop words, WordNet Lemmatizer and TF-IDF	Done
Supervised Learning	Multinomial Naive Bayes	Done
Evaluation on a separate set	80/20 split	Done
Binary classification of new messages	Predicts spam or Ham	Done
Evaluation from smaller to larger datasets	10%, 50% and 100% of the training data	Partly Done
Repo with the code and metrics	Public github repo : https://github.com/nathan-mersha/Project_nlp	Done

Table 9 : Requirement and Implementation checklist

6 Final thoughts and Conclusion

6.1 Reflection on the approach I took

The pipeline I chose for this classification worked really well. And it was easy to build with the available libraries. Because the approach I took was simple it was really easy to understand and explain each step. One thing I noticed is that when data is imbalanced accuracy alone is not really a good measure. For example the model trained on the 10% training data had an accuracy of 88% which sounds good, but in reality it's an almost useless model, it caught only 14 out of 149 spam messages in the test set.

6.2 Limitations

Even though the system works well, this project has some limitations that should be pointed out. Here are some of the limitations that I have observed :

- I have used only one data set. The evaluation from small to larger data was done with 10, 50 and 100 percent of the same SMS data set (which is around 4,457 messages). But not with different sets like email collection for example.
- I trained only one model (Multinomial Naive Bayes)
- The data sets are entirely English messages and these examples were published in 2011, and since spam message evolve overtime, a newer kind of spam may not be caught

6.3 Improvements

In the future the system could improve by testing it on bigger data sets like an email collection for example. And if more computing power is available, I could also try deep learning models as described by Elakkiya et al. (2021)

6.4 Conclusion

The goal of this project according to the task was to build an NLP system that sorts SMS messages into spam or legitimate. This goal has been reached successfully. The system takes SMS spam collections, cleans the text (by changing to lower case, splitting words, removing the stop words and lemmatizing it). It then turns the texts to numbers with TF-IDF and trains a Multinomial Naive Bayes classifier.

On a test of 1,115 messages, the final model reached an accuracy of 96.4%, a precision of 1.000, a recall of 0.732 and an F1 score of 0.845. The model never marked a legitimate message as spam at all, but missed some spams and marked them as legitimate. When we added more and more training data there was a clear improvement in the model. There are three takeaways from this project. First, a simple method that is very easy to understand can actually solve real world problems. Second, a very careful evaluation of the model with all the metrics that matter is as important as the model itself. Finally the amount of training data matters as seen in the example run, it’s clear that the more data we have the better. This was a really fun and applicable project to do and it’s one that can easily be improved.

Reference

Almeida, T. A., & Gómez Hidalgo, J. M. (2011). SMS spam collection [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C5CC84

Almeida, T. A., Gómez Hidalgo, J. M., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. In Proceedings of the 11th ACM Symposium on Document Engineering (pp. 259–262). ACM. https://doi.org/10.1145/2034691.2034742

Bird, S., Klein, E., & Loper, E. (2009). Natural language processing with Python. O'Reilly Media.

Elakkiya, E., Selvakumar, S., & Leela Velusamy, R. (2021). TextSpamDetector: Textual content based deep learning framework for social spam detection using conjoint attention mechanism. Journal of Ambient Intelligence and Humanized Computing, 12(10), 9287–9302.

Hopkins, M., Reeber, E., Forman, G., & Suermondt, J. (1999). Spambase [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C53G6X

Huan, H., Guo, Z., Cai, T., & He, Z. (2022). A text classification method based on a convolutional and bidirectional long short-term memory model. Connection Science, 34(1), 2108–2124.

Jurafsky, D., & Martin, J. H. (2013). Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition (2nd ed.). Pearson Prentice Hall.

Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to information retrieval. Cambridge University Press.

Metsis, V., Androutsopoulos, I., & Paliouras, G. (2006). Spam filtering with naive Bayes – Which naive Bayes? In Proceedings of the Third Conference on Email and Anti-Spam (CEAS 2006).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Sahami, M., Dumais, S., Heckerman, D., & Horvitz, E. (1998). A Bayesian approach to filtering junk e-mail. In Learning for text categorization: Papers from the 1998 workshop (AAAI Technical Report WS-98-05). AAAI Press.